Media Production and Asset Management Database Portfolio

Student	Samuel Oluwakoya Ipinlaiye
Student ID	301932362
Matriculation number	2026/A/MIT/0971
Programme	Master of Information Technology
Institution	Miva University
Course	MIT8103 - Advanced Database Systems

Project overview

This portfolio applies relational, NoSQL, transaction, optimisation, and distributed-database concepts to a fictional media production company. The organisation manages clients and projects, assigns employees and freelancers, catalogues media assets, and reserves production equipment. PostgreSQL 18.4 was used for structured operational data and MongoDB 8.2.12 for the document and distributed exercises. All data is deterministic and synthetic; media paths are simulated and no confidential or production records are included.

Component 1 - Database design and modelling

The relational model contains seven tables: `client`, `project`, `staff`, `project_assignment`, `media_asset`, `equipment`, and `equipment_reservation`. The Mermaid Crow's Foot diagram documents cardinality and optionality. The model is broadly in Third Normal Form: each entity is stored once, while the assignment table resolves the staff-project many-to-many relationship. This reduces duplication and update anomalies, such as repeating client contact details in every project.

Generated integer primary keys provide stable identifiers. Eight foreign keys protect relationships and restrictive deletion preserves history. Check constraints validate statuses, dates, budgets, versions, sizes, and reservation periods. A partial unique index on `lower(email)` makes supplied staff emails case-insensitively unique while allowing multiple `NULL` values. A trigger prevents inactive staff from creating reservations. The `btree_gist`-based exclusion constraint blocks overlapping `confirmed` or `checked_out` reservations for the same equipment, while permitting provisional, historical, and back-to-back bookings. The rollback-only validation suite passed all 11 tests. The ERD, DDL, rationale, and output are under `component-1-design/` and `evidence/component-1/`.

Component 2 - Query processing and optimisation

A reproducible `generate_series` loader produced 766,500 rows across the seven tables: 500 clients, 10,000 projects, 1,000 staff, 50,000 assignments, 500,000 assets, 5,000 equipment records, and 200,000 reservations. Three operational queries reported assets awaiting review, current staff workload, and a confirmed equipment schedule. Each query received one warm-up followed by five genuine `EXPLAIN (ANALYZE, BUFFERS, SETTINGS)` executions before and after candidate indexes, without forcing planner choices.

Query 1's median fell from **24.040 ms to 6.572 ms**. Its covering media-asset index changed a parallel sequential scan into an index-only scan and reduced root-level shared-buffer activity in the median-time plans from 19,531 buffer accesses (11,662 hits and 7,869 reads) to 367 hits. Query 2 changed from **17.116 ms to 17.795 ms**. PostgreSQL correctly retained a sequential scan because much of the assignment table was needed; its candidate index was unused, so this result is not presented as an improvement. Query 3's median fell from **13.577 ms to 3.765 ms**, changing a parallel sequential scan into an index-only scan and reducing shared-buffer hits from 3,282 to 544. Complete plans and the five-run values are under `evidence/component-2/`.

Component 3 - Transactions and concurrency

Explicit transactions demonstrated that an inserted client disappeared after `ROLLBACK`, whereas a second record remained visible after `COMMIT` until its targeted cleanup. A two-session reservation experiment then tested the exclusion constraint. When Session A held an overlapping confirmed reservation and later committed, Session B waited about 4.49 seconds on a transaction-ID lock and then received SQLSTATE `23P01`. When Session A instead rolled back, Session B waited about 4.48 seconds and completed successfully before its own rollback. A third connection captured `pg_stat_activity`, `pg_locks`, and `pg_blocking_pids()` evidence during both waits.

The isolation comparison used a temporary project budget. Under `READ COMMITTED`, Session A read 100000.00 and then saw Session B's committed 110000.00 on its next statement. Under `REPEATABLE READ`, Session A continued to see 100000.00 inside the same transaction after Session B committed 120000.00, then saw 120000.00 after committing and

starting a new statement. Targeted cleanup removed every fixture and restored all seven baseline counts. Scripts and multi-session output are under `component-3-transactions/` and `evidence/component-3/`.

Component 4 - NoSQL and advanced data models

MongoDB was deployed through Docker Compose using the pinned `mongo:8.2.12-noble` image after compatibility testing on the ARM64 Docker Desktop environment. The project-centred design uses five collections: `projects`, `staff`, `mediaAssets`, `equipment`, and `equipmentReservations`. Small, bounded values such as client snapshots, crew, tags, and media versions are embedded where they support aggregate reads. Potentially unbounded asset and reservation histories remain separate and use references. This contrasts with PostgreSQL's normalised tables and enforced foreign keys; MongoDB gains flexible nested reads but requires explicit reference and snapshot validation.

The deterministic dataset contains **12,350 documents**. All five collections have strict validators, and verification found no missing references, snapshot mismatches, invalid periods, inactive bookers, array-bound violations, or remaining CRUD fixtures. Demonstrations covered create, read, update, delete, and four aggregations. For the indexed review-asset query, the winning access path changed from `COLLSCAN` plus in-memory sorting to `IXSCAN`. Documents examined fell from **10,000 to 554**, with the same 554 results. Reported time changed from **5 ms to 4 ms**; this one-millisecond difference is not treated as strong performance evidence because the dataset is small and MongoDB reports integer milliseconds. Evidence is under `evidence/component-4/`.

Component 5 - Distributed and cloud database concepts

A separate authenticated three-node MongoDB replica set used an internal Docker network, unique volumes, shared keyfile member authentication, and no published database ports. It is a **local distributed deployment, not a cloud deployment**. Initially, `mongo1` was primary and `mongo2` and `mongo3` were secondaries. A majority write inserted 30 deterministic project documents and authenticated reads confirmed the same count and IDs on every member.

After the current primary stopped, `mongo2` was observed as the new primary approximately four seconds later. It accepted `DPRJ-0031` with explicit majority write concern. The former primary restarted and rejoined as a healthy secondary; all three members then returned the same 31 IDs. Replication provides redundant copies and election-based failover, while majority acknowledgement trades some latency and write availability for stronger durability. Secondary reads can improve read availability but may be stale. Managed cloud services use related mechanisms while additionally offering provider-operated multi-zone placement, backups, monitoring, and scaling. Status, election, write, and final consistency evidence are under `evidence/component-5/`.

Limitations and reproducibility

All experiments ran on one Apple Silicon computer with synthetic data. Local warm-cache timings do not predict production performance, and deterministic distributions can influence planner estimates. MongoDB's reservation index does not reproduce PostgreSQL's cross-row exclusion guarantee. The replica-set nodes share one Docker host, so this exercise does not demonstrate geographical fault tolerance, cloud-provider service levels, sharding, managed backup, or elastic scaling.

Scripts, execution order, prerequisites, and safety conditions are documented in the root README and each component. Raw evidence includes genuine diagnostic failures as well as successful final runs. Repeating experiments can change timings, PIDs, transaction IDs, buffer state, and election duration.

AI-use disclosure

AI assisted with planning, implementation, debugging, analysis, and documentation. Database claims and numerical results were verified through actual PostgreSQL and MongoDB execution and retained terminal evidence. The portfolio does not claim that every script or explanation was independently authored by the student. The student is responsible for reviewing, understanding, and accurately presenting the submitted work in accordance with university policy.

Personal Technical Reflection

This portfolio strengthened my understanding of how database design decisions affect performance, consistency, and availability. Working with PostgreSQL demonstrated the importance of relational integrity, indexing, transaction isolation, and concurrency control. One particularly useful observation was that adding an index does not automatically improve query performance, as demonstrated by the staff workload query.

The MongoDB exercises helped me understand the differences between relational and document-oriented data models. I also gained practical experience with distributed database replication, majority writes, primary elections, and failover. Resolving Docker and MongoDB compatibility issues reinforced the importance of environment verification and systematic troubleshooting.

Overall, the exercise connected database concepts to practical implementation decisions in a media production and asset management system.